Aris Ozark, Dylan Thornburg

ECEN 133L-1

Thursday, May 29th - 2:15 to 5:00

Lab 8 - Week 9

Lab #8: Adaptive Systems

Objective:

The purpose of this lab is to experiment with adaptive and least squares filtering. We will investigate the roles of filter size, learning rates, and evaluating susceptibility to noise. Also, we will experiment with identifying various FIR filters and optimizing zero-valued weights to fit them over many samples.

Part 1:

Following the directions in the lab document, we produced a MATLAB adaptive filter script which returned the following results:

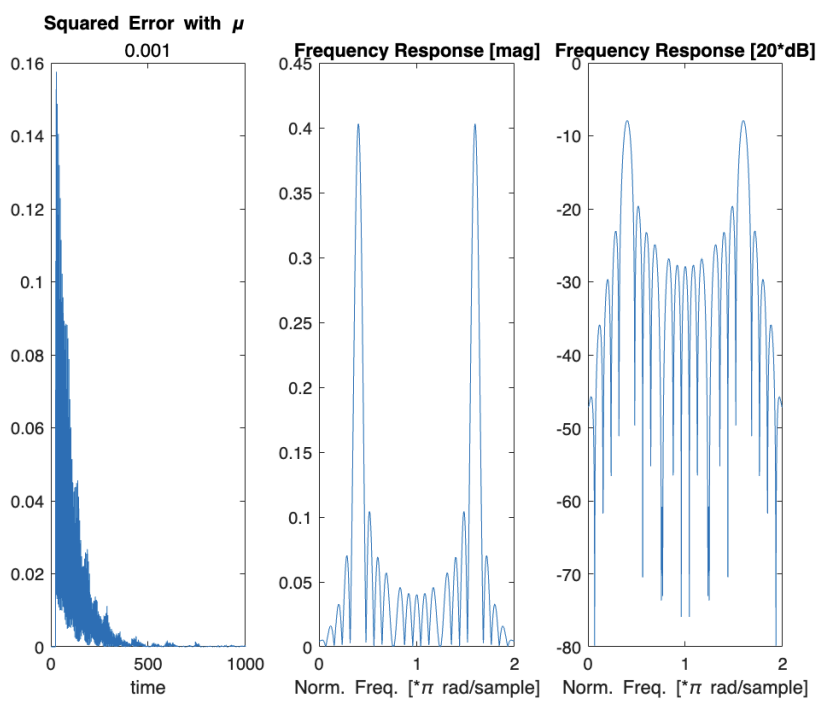

Figure 1: Adaptive filter with no noise, mu = 0.001

It is observed that the squared error quickly converges to near-zero after roughly 500 trials.

Additionally, the frequency response clearly shows that the filter effectively minimized frequencies outside 0.4*pi radians/sample

Part 2:

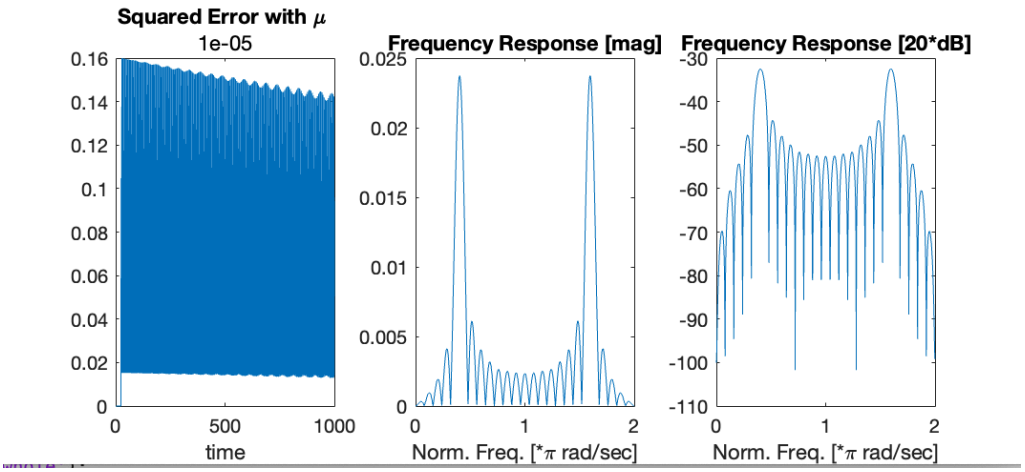

Figure 2: Adaptive filter with noise coefficient 0.1, mu = 10^-5

From Figure 2, the lowest value of the squared error is 0.013. The squared error rapidly oscillates, but it reaches a near-zero value after roughly 800 iterations. The frequency response looks very similar to the trial filter, except there are more lobes, and the maximum fft magnitude is only 0.24. For comparison, the identical least-squares scenario is shown below:

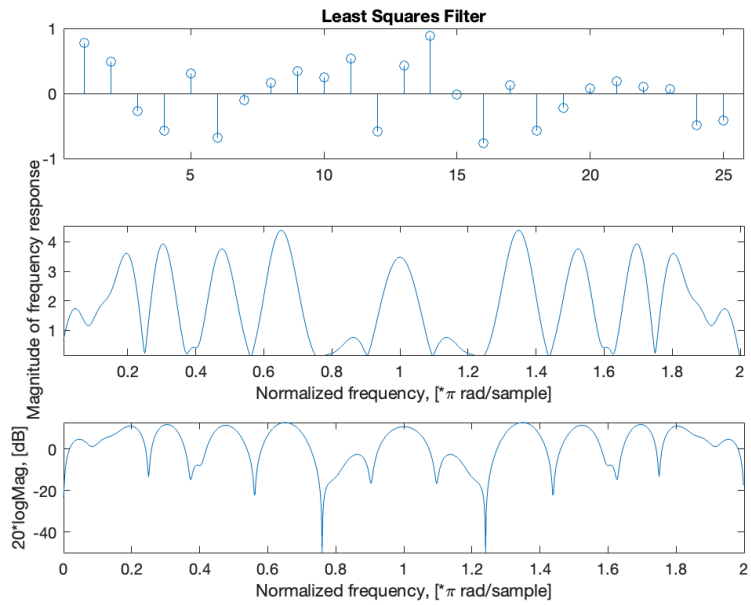

Figure 3: Least squares filter

Compared to the least squares filter, the adaptive filter is far neater and better-defined. Moreover, MATLAB gave a warning that the autocorrelation matrix was nearly non-singular, meaning that we were approaching the edge of stability. This is another reason why adaptive filtering over a smaller subset of the input data over processing all at once is more desirable.

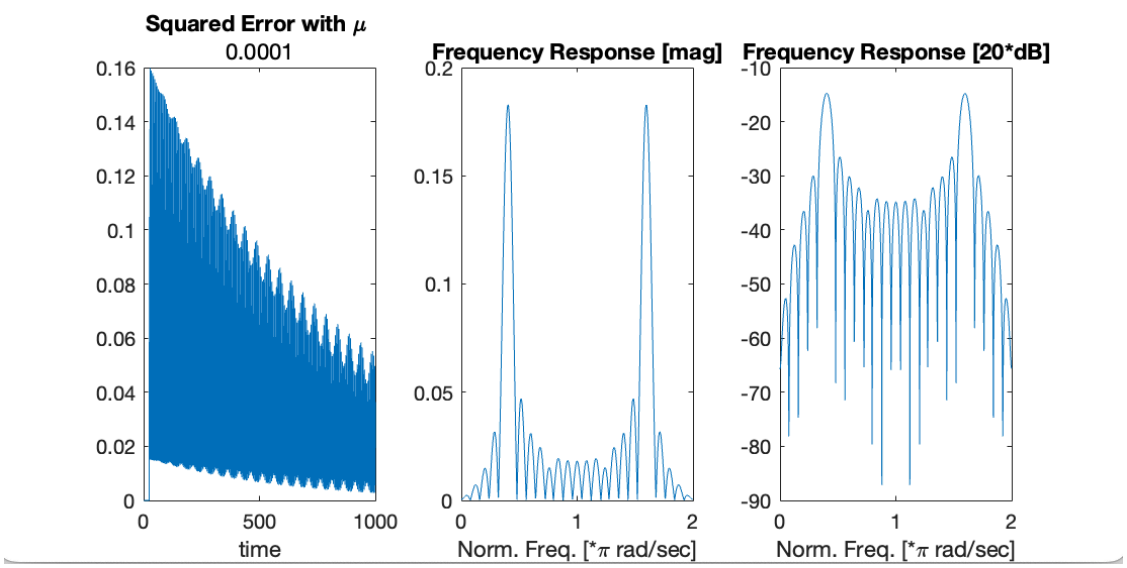

Figure 4: Adaptive filter, mu = 0.0001

What is the lowest value of the squared error?
0.027

If the error goes to close to zero, how many iterations does it take?
~700

After L data samples how much does the shape of the fft of the filter look like the least squares filter fft from the class lecture slides?
Looks very similar except you could say there's more lobes.

What is the maximum value of the adaptive filter's fft magnitude?
.18

How does that compare to the least squares fft?
Looks a lot cleaner.

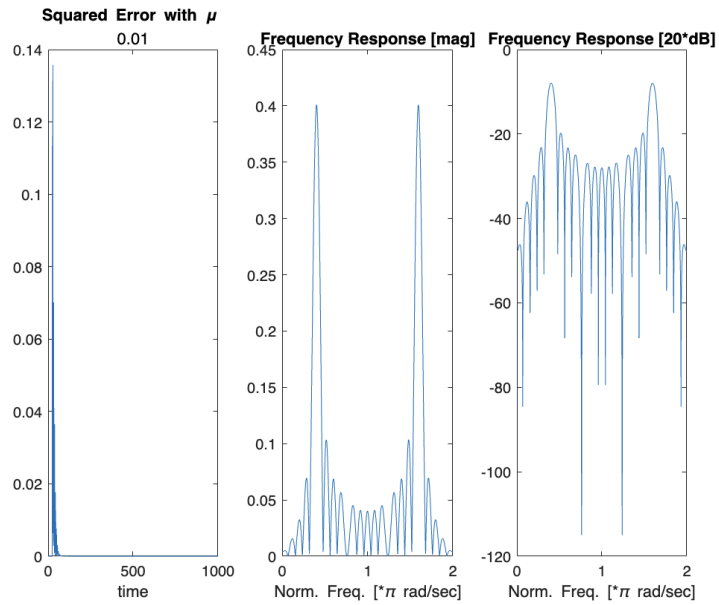

Figure 5: Adaptive filter, mu = 0.01

What is the lowest value of the squared error?
0

If the error goes to close to zero, how many iterations does it take?
~100

After L data samples how much does the shape of the fft of the filter look like the least squares filter fft from the class lecture slides?
Looks very similar except you could say there's more lobes.

What is the maximum value of the adaptive filter's fft magnitude?
.4

How does that compare to the least squares fft?
Looks a lot cleaner.

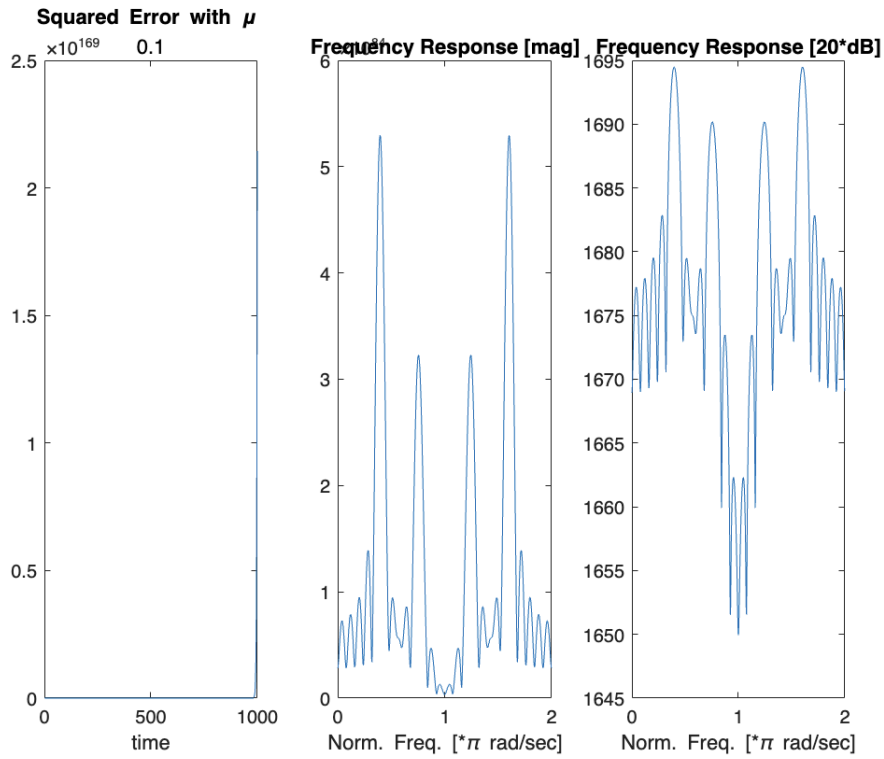

Figure 6: Adaptive filter, mu = 0.1

What is the lowest value of the squared error?
4.412e128 (int limit)

If the error goes to close to zero, how many iterations does it take?
The filter never goes to zero, in fact, its minimum observed value is well over 10^96. The learning rate is too large, and it is never able to locate the minimums within the parameter space.

After L data samples how much does the shape of the fft of the filter look like the least squares filter fft from the class lecture slides?
Looks very similar except you could say there's more lobes.

What is the maximum value of the adaptive filter's fft magnitude?
5.27e84

How does that compare to the least squares fft?
Least squares filter actually exists so it is better.

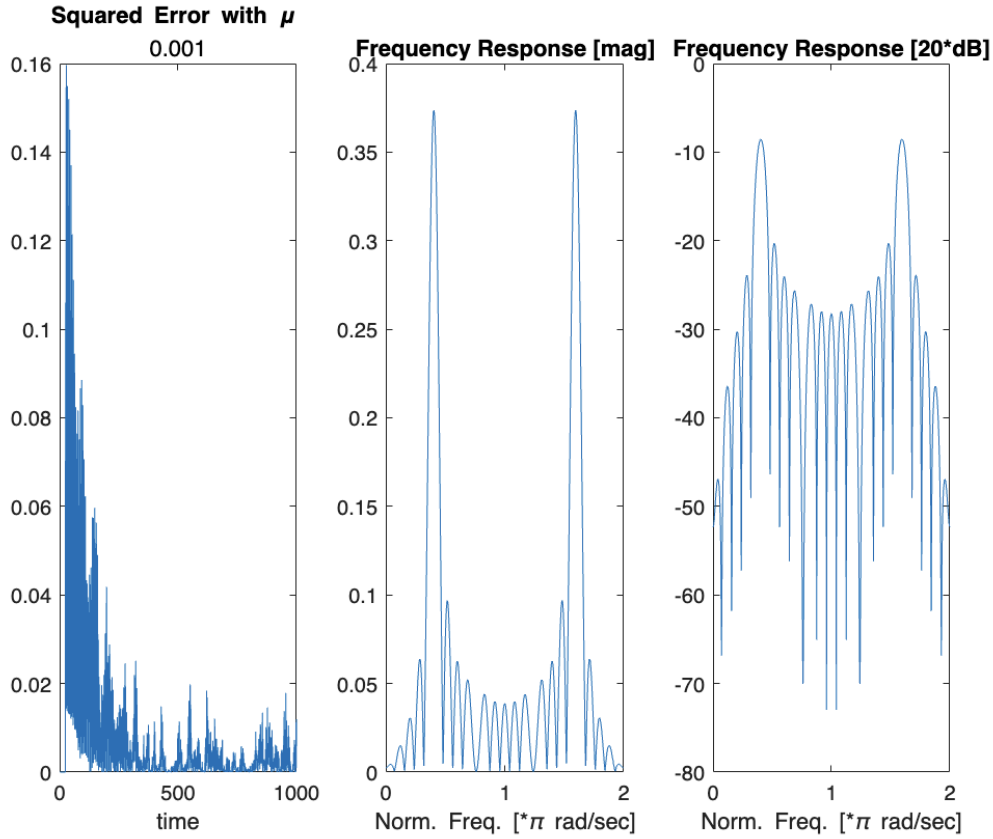

Figure 7: Adaptive filter, mu = 0.1, noise constant 0.5

How do these results compare to Step 1 results? Why do you think the squared error curve is different?

The squared error curve doesn't converge to 0 as quickly because there is more noise to deal with.

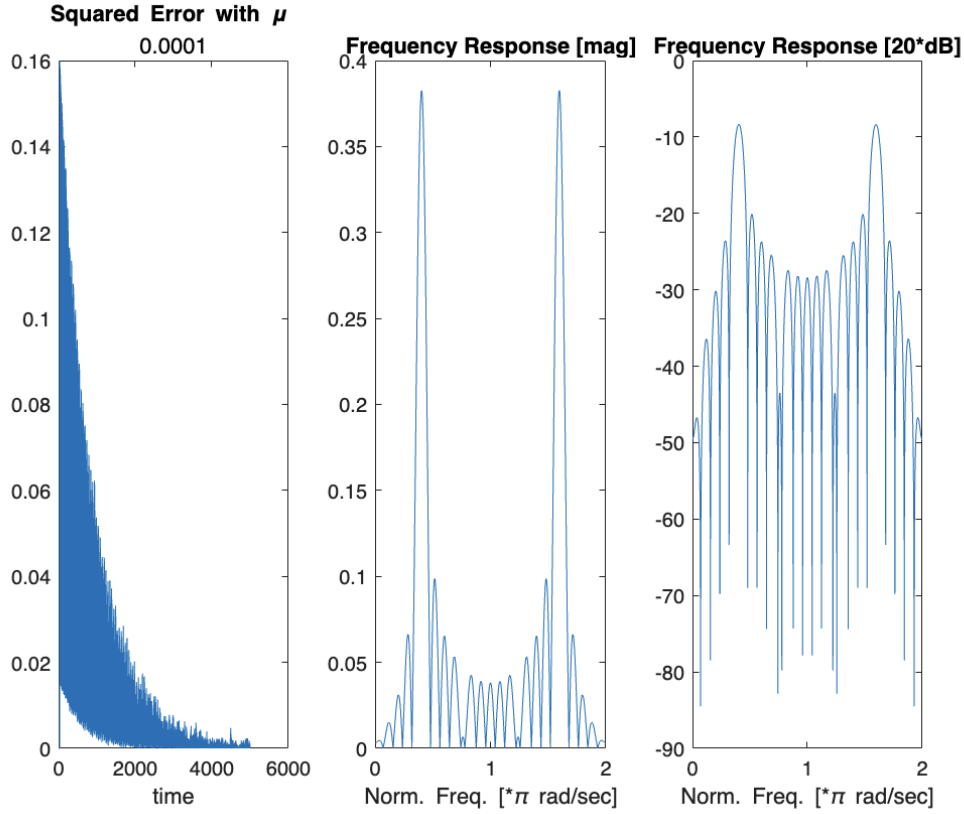

Figure 8: Adaptive filter, mu = 0.0001, noise constant 0.1

Compare the results to the result earlier in this step which had the same value of mu and a, but had L = 1000.

It is the same error curve, just extended. There's more time for the filter to converge the error to 0.

Part 3:

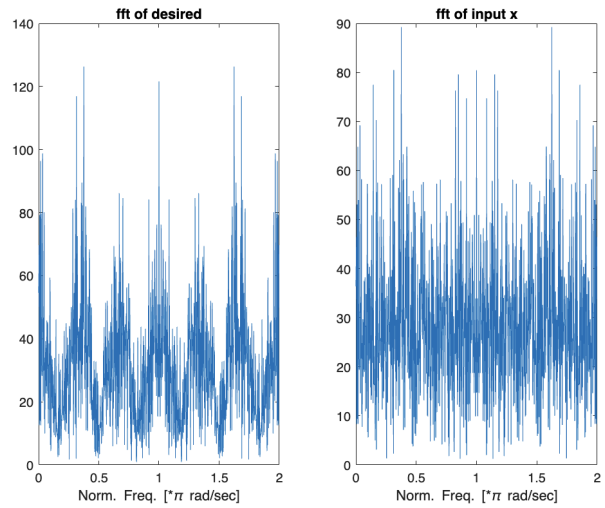

Figure 9: fft of desired versus input noise signal

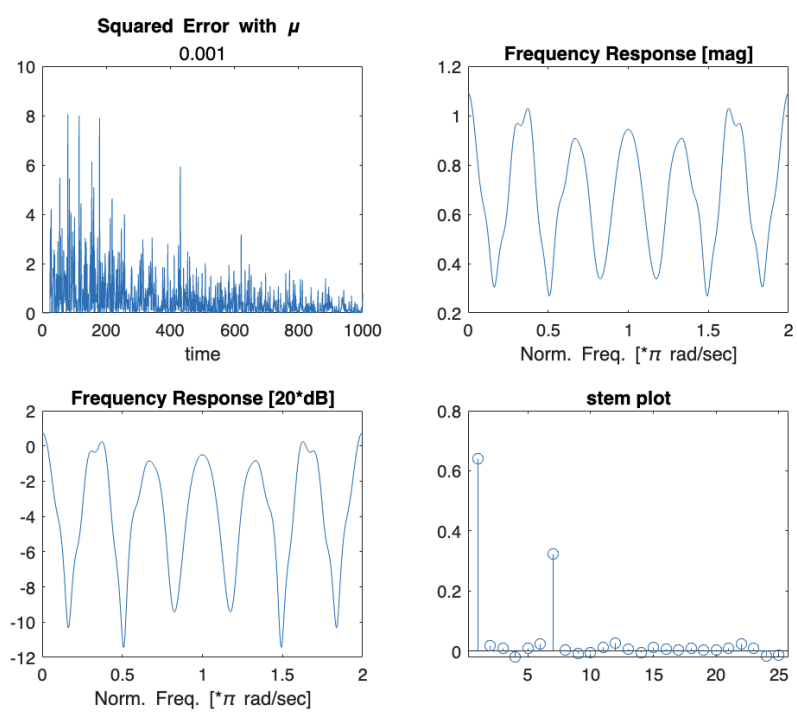

Figure 10: Adaptive filter, to unknown system, learning rate 0.001

Compare the filter w with the unknown filter h. How similar is it?
Somewhat similar, but the filter w is better. Filter h's output had .64 and .32 instead of 1 and .5. The learning rate is too small for the filter's coefficients to increase to the desired values.

What is the error level at the end of the adaptation loop?
About .22

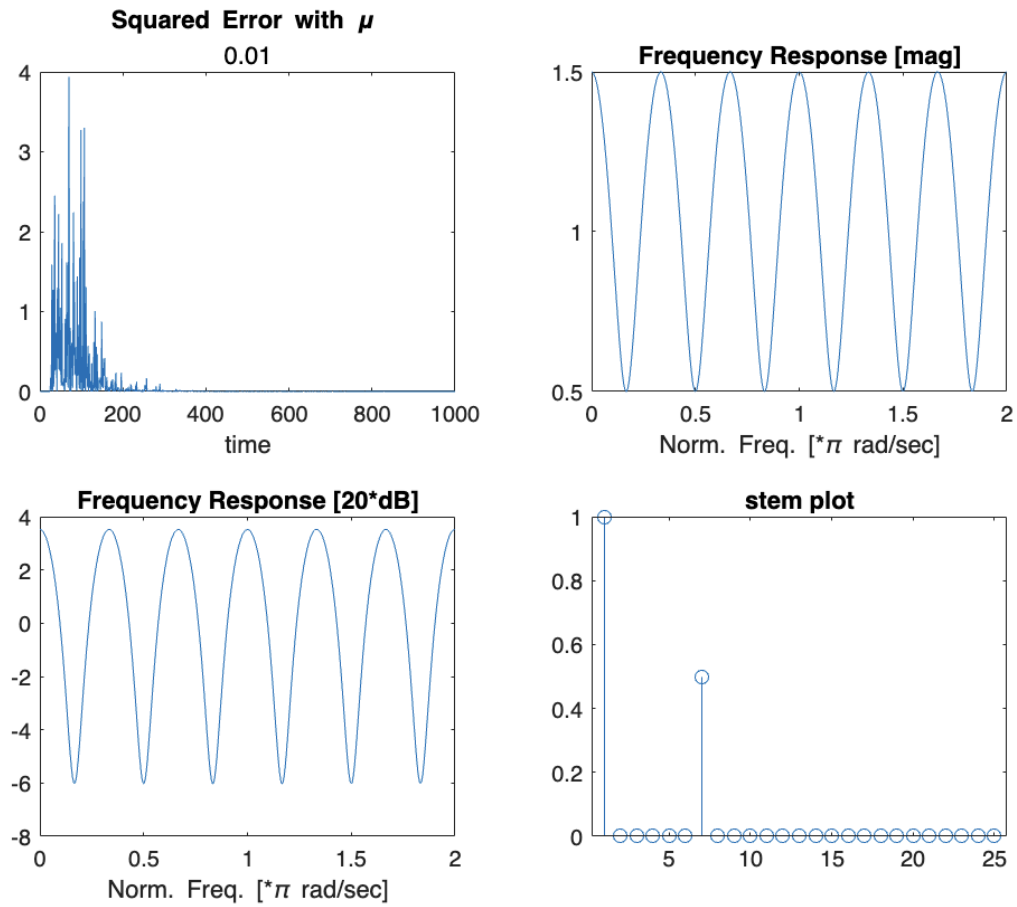

Figure 11: Adaptive filter, to unknown system, learning rate 0.01

This unknown filter works very well and is able to conform to the data very quickly due to the increased learning rate.

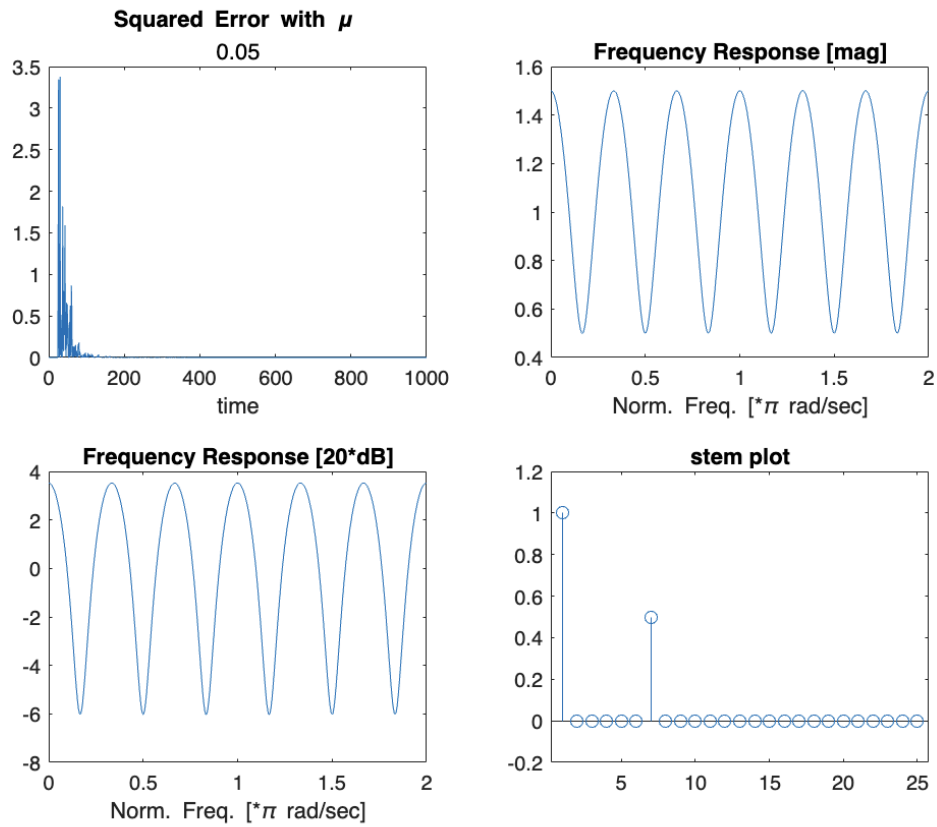

Figure 12: Adaptive filter, to unknown system, learning rate 0.05

Mu of .05 converges error to zero quicker than .01. This mu still works well.

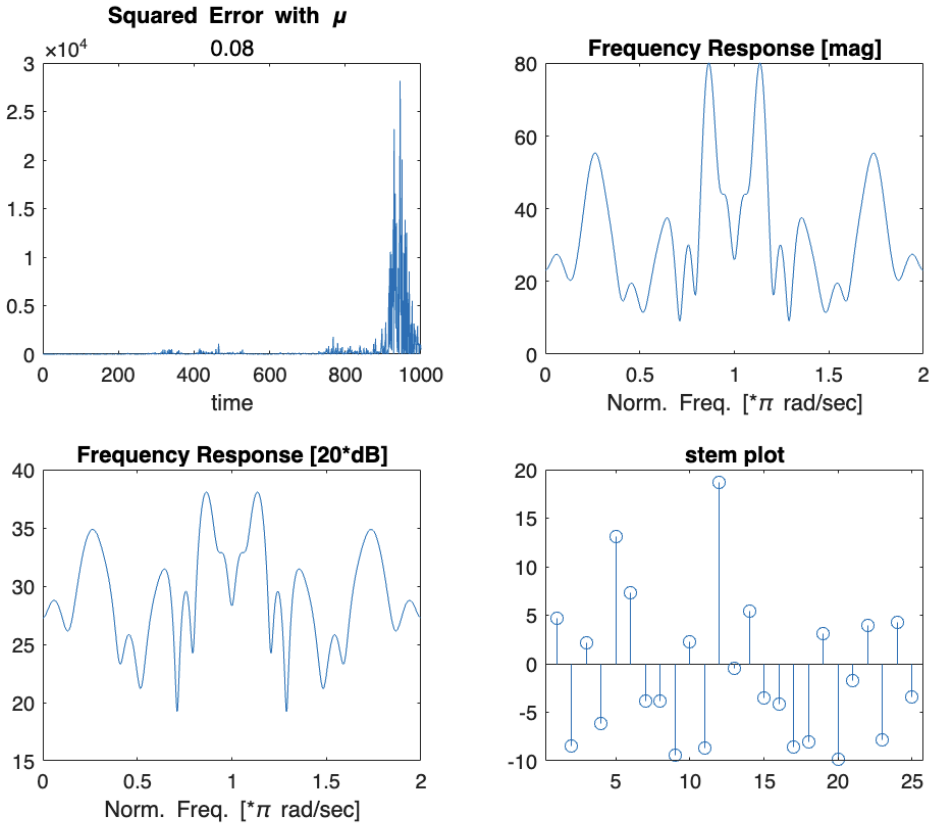

Figure 13: Adaptive filter, to unknown system, learning rate 0.08

Now we see the mu is too high. The adaptation is moving too fast creating an exploding gradient. It's not as bad as the mu=.1.

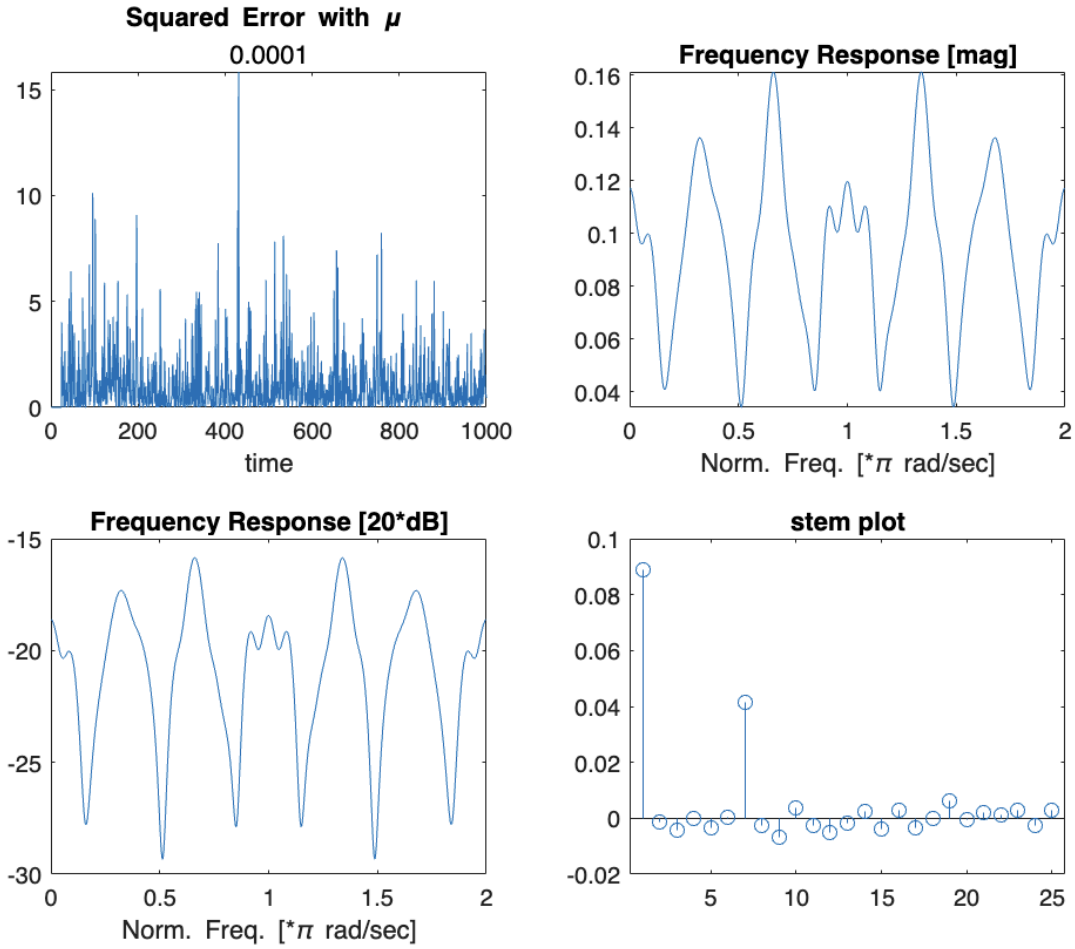

Figure 14: Adaptive filter, to unknown system, learning rate 0.0001

This mu doesn't change fast enough. It is not a good option.

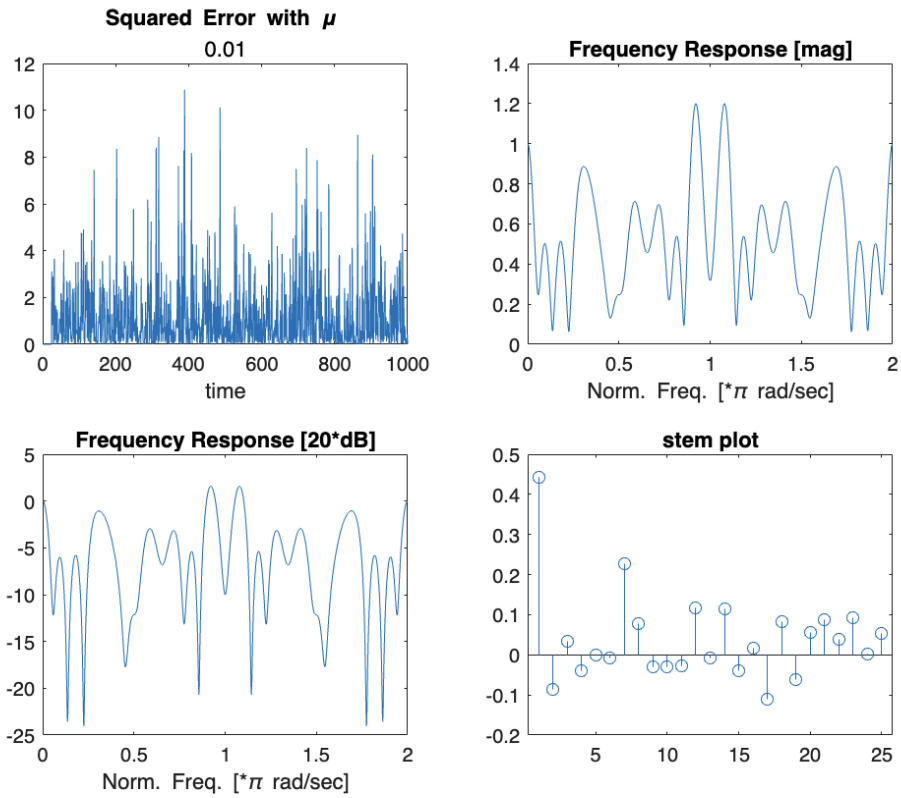

Figure 15: Adaptive filter, to unknown system, learning rate 0.01

The out of phaseness of d renders the filter pretty much useless. Since the desired filter is delayed from the system trying to mimic it, it will always be out of step and trying to adapt to things that have already occurred. Thus it will never converge.

Part 4:

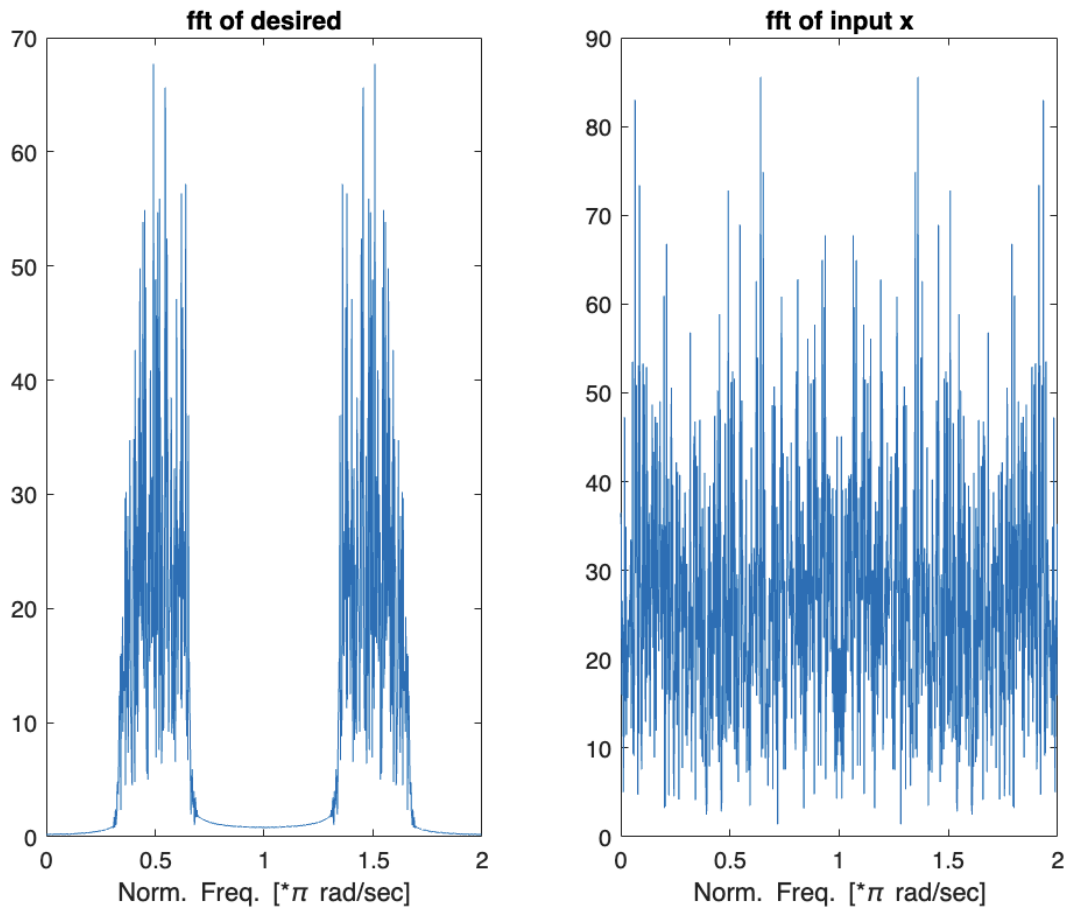

Figure 16: Parks-McClellan filter fft versus random noise input, 64th order

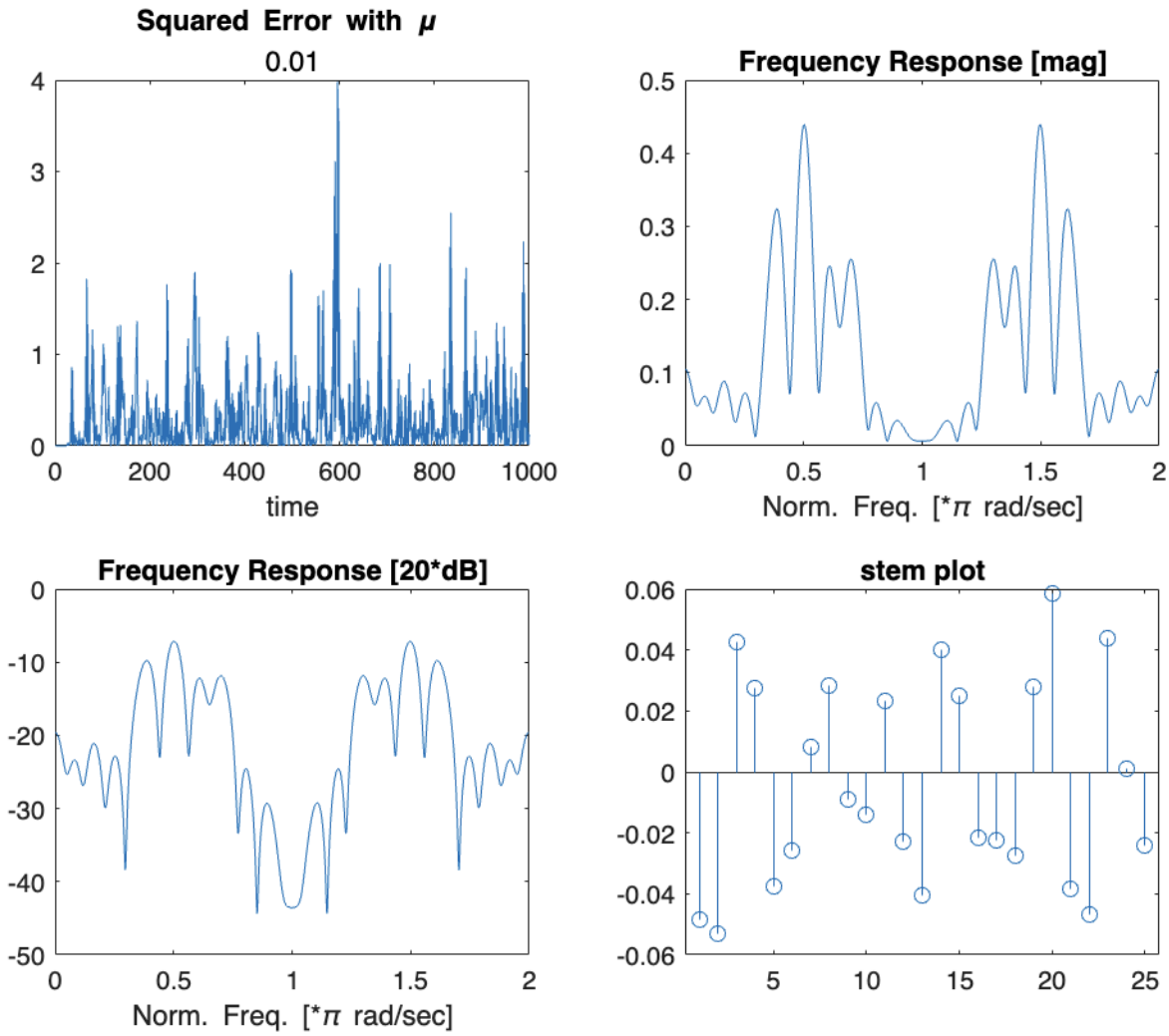

Figure 17: Parks-McClellan adaptation, 0.01

Does the error get reduced as the filter adapts?
No it doesn't because the filter isn't long enough (25 vs 64) to capture all the weights.

Try a few other values for mu and see if there is any improvement.
The mu didn't matter. We're still short for error.

Why is this filter not working well?
The filter doesn't have a big enough order, so it can't capture what's going on.

Make a stem plot of the unknown filter b_pm and compare it to the stem plot or your filter w.

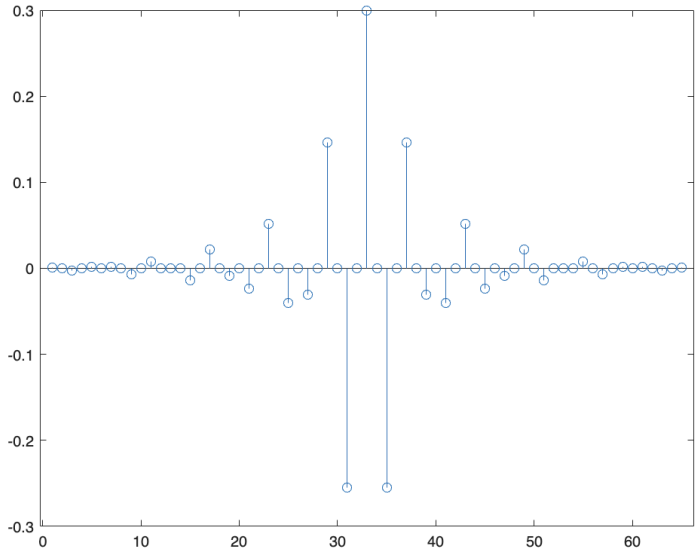

Figure 18: Parks-McClellan ideal impulse response, 64th order

As can be seen, the 64th order filter cannot be captured by an adaptive filter of length 25.

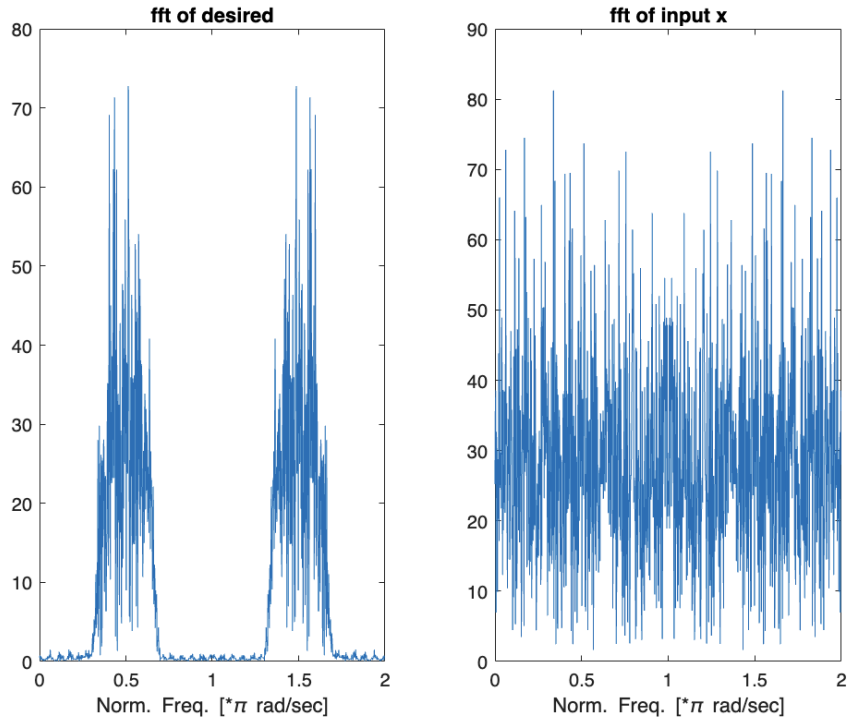

Figure 19:Parks- McClellan fft, 32nd order

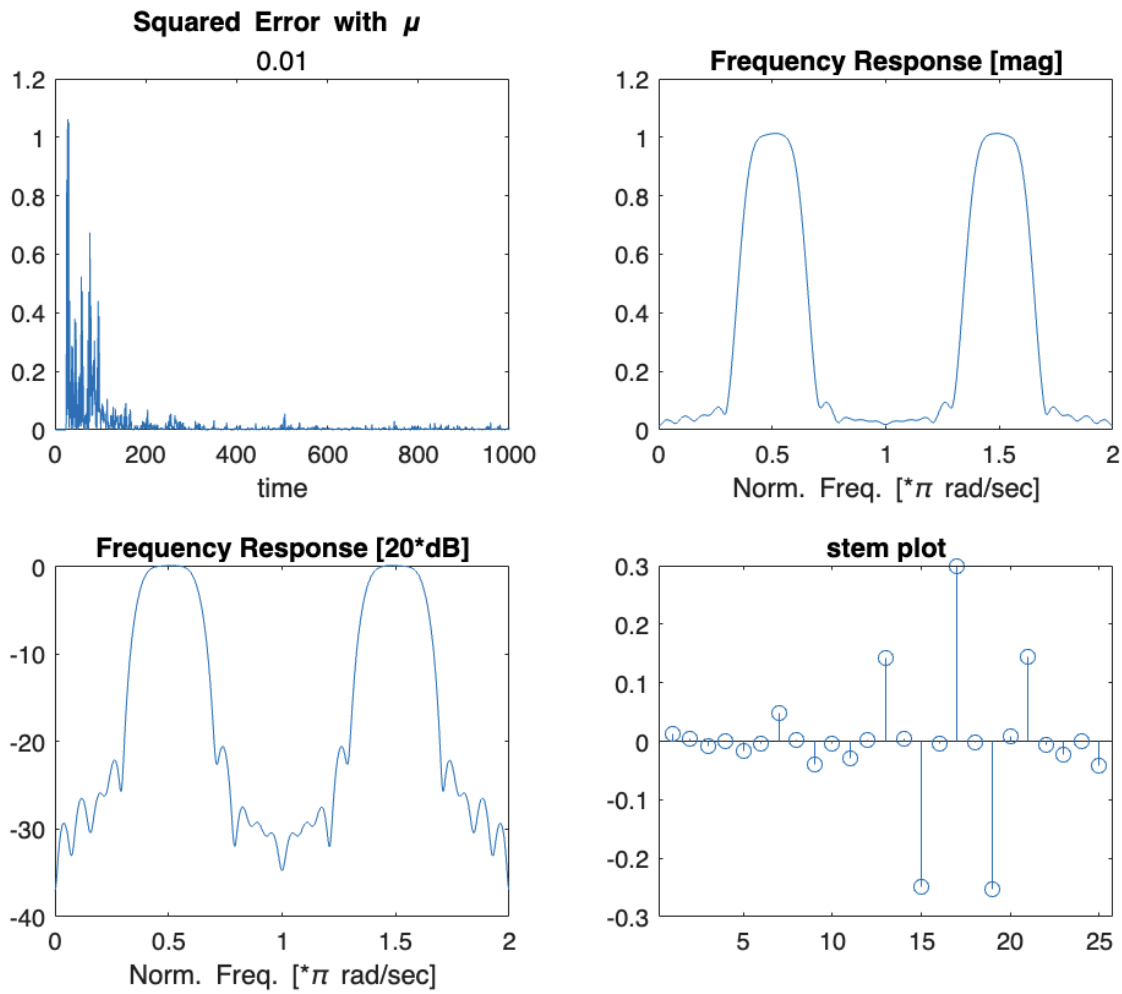

Figure 20: Parks-McClellan adaptation, 0.01, 25th order

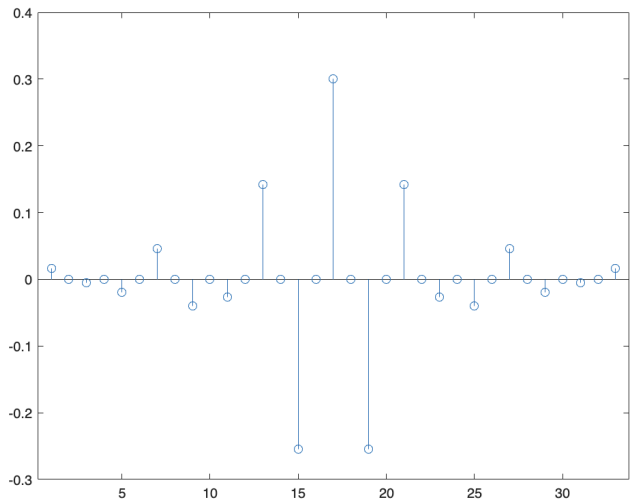

Figure 21: Parks-McClellan impulse response, 32nd order

Repeat the steps above but change n_pm from 64 to 32 and use mu = 0.01. Explain why this change of n_pm made an improvement compared to the previous result. What other change might have improved the result?

This n_pm now captures a lot more of the weights and thus the error can converge. We could also increase the filter length (M) to 32 to improve the result. Overall, the filter response is far better when the weights can actually represent most of the desired filter

Conclusions:

In this lab, we gained experience working with adaptive and least squares filters. One important takeaway is that adaptive filtering is often better than direct least squares because one can gradually approach the weights with a simple computation instead of performing an inversion operation on a big, unwieldy, possibly unstable matrix. The downside is that there are more parameters to optimize and consider, for instance learning rate. Another key takeaway is that when trying to identify/mimic an unknown system, you may need to experiment with different filter sizes to ensure the filter can accurately capture the length of the filter and converge properly. Overall, this lab was useful for gaining intuition for adaptive systems.